

Reviewer Pack — Forman-Ricci Min-Curvature of Term-Overlap Graph Lower-Bound...

Entry b0a4fb5d3039 · FALSIFIED

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 22:25:13 UTC

Forman-Ricci Min-Curvature of Term-Overlap Graph Lower-Bounds Monotone k-CLIQUE DNF

Entry ID: b0a4fb5d3039 Verdict: FALSIFIED Recorded: 2026-05-17 22:25:13 UTC

1. Conjecture

Field A (mathematical branch): Forman’s combinatorial Ricci curvature on weighted 1-skeleta (Forman 2003, ‘Bochner’s method for cell complexes and combinatorial Ricci curvature’; Sreejith-Mukherjee-Sandhu-Saucan-Jost 2016 on network curvature). Distinct from Ollivier-Ricci (Skenderi 2025): Forman uses parallel-face / Bochner-Weitzenböck local sums of vertex and edge weights, not Wasserstein optimal transport. An arXiv search for ‘Forman Ricci’ AND (‘monotone circuit’ OR ‘DNF’ OR ‘proof complexity’ OR ‘Razborov’) returns 0 direct hits and <5 adjacent papers, all on biological/social networks — never on Boolean-function DNFs. **Field B** (complexity object): Monotone DNF leaf size / formula size for the k-CLIQUE indicator on K_v ($n = v(v-1)/2$ edge variables) in the Razborov 1985 / Alon-Boppana / Andreev regime; the Cook-Reckhow-Krajicek bounded-arithmetic stepping stone where a submodular DNF measure that is small on poly-size DNFs but $\Omega(v)$ on k-CLIQUE would witness a V^0 -style separation by Buss-Pudlak-style approximator counting.

Statement:

For a monotone DNF $F = \bigvee_{i=1}^s T_i$ (terms $T_i \subseteq [N]$) with at least one term of size ≥ 1 , build the weighted term-overlap graph G_F on vertex set $\{1, \dots, s\}$: place an edge ij iff $|T_i \cap T_j| \geq 1$, assign vertex weight $w_i = |T_i|$ and edge weight $w_{ij} = |T_i \cap T_j|$, and define the Forman-Ricci curvature $\text{Ric}_F(ij) = w_{ij} \cdot [w_i/w_{ij} + w_j/w_{ij} - \sum_{k \sim i, k \neq j} w_i/(w_{ij} \cdot w_{ik}) - \sum_{k \sim j, k \neq i} w_j/(w_{ij} \cdot w_{jk})]$. Let $\mu(F) := \log_2(1 + \max\{0, -\min_e \text{Ric}_F(e)\})$ (with $\mu = 0$ if G_F has no edges). We conjecture: (i) for any pair of monotone DNFs F, G , $\mu(F \wedge G) \leq \mu(F) + \mu(G) + \log_2(1 + N)$ (approximate submodularity under conjunction); (ii) for every monotone DNF F with $|F| \leq N^c$ terms, $\mu(F) \leq 6c \cdot \log_2(1 + N)$; and (iii) the canonical minterm DNF F^*_v of the k-CLIQUE indicator on K_v with $k = \lceil \log_2 v \rceil$ satisfies $\mu(F^*_v) \geq v/4$. A single instance violating any of (i),(ii),(iii) refutes the conjecture.

Rationale (proposer’s reasoning):

The Razborov approximator method replaces $\wedge$ and v with closed operations on ‘small CC^k objects’ and bounds the error count; the right measure must be submodular under $\wedge$ and saturate on clique. Forman-Ricci collapses each term-overlap edge into a single weighted Bochner-Weitzenböck term, so global negativity of the minimum curvature is forced precisely when the term graph contains a high-degree dense neighborhood around a ‘sunflower core’ — exactly the structural obstruction that makes clique indicators hard to approximate. Because μ is computed on the DNF graph (not the truth table) and

uses $\sqrt{\cdot}$ -weighted sums (not ring polynomial extensions), it is shielded from both NATURAL_PROOFS and ALGEBRIZATION.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f465cd6fee8eadfb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds per regime (A-D), conjecture is SUPPORTED iff $\geq 95\%$ of all instances satisfy their respective inequality AND every K_v in regime (A) satisfies $\mu(F^*_v) \geq v/4$ with mean slack ≥ 0 across seeds. Any single hard falsifier in (A),(ii)-bound in (B), or (i)-bound in (D) refutes.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forman Ricci curvature monotone DNF circuit lower bound - combinatorial Ricci curvature Boolean function complexity measure clique - term overlap hypergraph curvature submodular approximator Razborov

Top relevant hits considered: - [<http://arxiv.org/abs/2103.10093v1>] Conjectures on Convergence and Scalar Curvature - [<http://arxiv.org/abs/2403.15973v1>] Isoperimetric profile function comparisons with Integral Ricci curvature bounds - [<http://arxiv.org/abs/1809.07698v1>] Discrete Ricci curvatures for directed networks - [<http://arxiv.org/abs/2304.02672v2>] The Log-Sobolev inequality for a submanifold in manifolds with asymptotic non-negative intermediate Ricci curvature - [<http://arxiv.org/abs/1909.00234v1>] The spectrum of a class of uniform hypergraphs - [<http://arxiv.org/abs/1909.00246v2>] The Signless Laplacian Matrix of Hypergraphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict
```

```

def generate_clique_dnf(v, k):
    terms = []
    for i in range(v):
        term = set()
        for j in range(i, v):
            if j != i:
                term.add((i, j))
            if len(term) >= k:
                terms.append(term)
    return terms

def generate_random_dnf(N, s, k):
    terms = []
    for _ in range(s):
        term = set(random.sample(range(N), k))
        terms.append(term)
    return terms

def generate_simple_dnf(N, target):
    if target == "THRESHOLD-2":
        terms = [set([i]) for i in range(N)]
    elif target == "AND":
        terms = [set(range(N))]
    elif target == "OR":
        terms = [set([i]) for i in range(N)]
    elif target == "MAJ-on-5":
        terms = [set([i]) for i in range(5)]
    elif target == "k=2-CLIQUE":
        terms = [set([i, j]) for i in range(N) for j in range(i+1, N)]
    else:
        raise ValueError("Unknown target")
    return terms

def build_term_overlap_graph(terms):
    graph = defaultdict(dict)
    for i, term_i in enumerate(terms):
        for j, term_j in enumerate(terms):
            if i != j:
                intersection = len(term_i & term_j)
                if intersection >= 1:
                    graph[i][j] = intersection
    return graph

def compute_forman_ricci_curvature(graph, terms):
    ricci = {}
    for i in graph:
        for j in graph[i]:
            if (j, i) not in ricci:
                w_ij = graph[i][j]
                w_i = len(terms[i])
                w_j = len(terms[j])
                sum_i = sum(w_i / math.sqrt(w_ij * graph[i][k]) for k in graph[i] if k != j)
                sum_j = sum(w_j / math.sqrt(w_ij * graph[j][k]) for k in graph[j] if k != i)
                ricci[(i, j)] = w_ij * (w_i / w_ij + w_j / w_ij - sum_i - sum_j)
    return ricci

```

```

def compute_mu(ricci):
    if not ricci:
        return 0
    min_curvature = min(ricci.values())
    return math.log2(1 + max(0, -min_curvature))

def run_trial(seed):
    random.seed(seed)
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
    metric_values = []

    # Regime A: Canonical minterm DNF for k-CLIQUE
    for v in [4, 5, 6, 7, 8]:
        k = math.ceil(math.log2(v))
        terms = generate_clique_dnf(v, k)
        graph = build_term_overlap_graph(terms)
        ricci = compute_forman_ricci_curvature(graph, terms)
        mu = compute_mu(ricci)
        metric_values.append(mu)
        instances_tested += 1
        if mu < v / 4:
            conjecture_holds = False
            counterexample = f"Regime A: v={v}, mu={mu} < v/4={v/4}"
            break

    if conjecture_holds:
        # Regime B: Simple monotone DNFs
        targets = ["THRESHOLD-2", "AND", "OR", "MAJ-on-5", "k=2-CLIQUE"]
        for target in targets:
            N = 36
            terms = generate_simple_dnf(N, target)
            graph = build_term_overlap_graph(terms)
            ricci = compute_forman_ricci_curvature(graph, terms)
            mu = compute_mu(ricci)
            metric_values.append(mu)
            instances_tested += 1
            c = math.log2(len(terms)) / math.log2(N) if len(terms) > 1 else 1
            if mu > 6 * c * math.log2(1 + N):
                conjecture_holds = False
                counterexample = f"Regime B: target={target}, mu={mu} > 6c·log2(1+N)={6*c*math.log2(1+N)}"
                break

    if conjecture_holds:
        # Regime C: Random monotone DNFs
        N = 36
        for s in [N, N * math.log2(N), N ** 1.5]:
            for k in [3, 4, 5]:
                terms = generate_random_dnf(N, int(s), k)
                graph = build_term_overlap_graph(terms)
                ricci = compute_forman_ricci_curvature(graph, terms)
                mu = compute_mu(ricci)
                metric_values.append(mu)
                instances_tested += 1

```

```

if conjecture_holds:
    # Regime D: Pairs of DNFs
    N = 36
    for _ in range(5):
        terms1 = generate_random_dnf(N, N, 3)
        terms2 = generate_random_dnf(N, N, 3)
        terms_conj = [t1 & t2 for t1 in terms1 for t2 in terms2 if t1 & t2]
        if len(terms_conj) > 10000:
            terms_conj = terms_conj[:10000]
        graph_conj = build_term_overlap_graph(terms_conj)
        ricci_conj = compute_forman_ricci_curvature(graph_conj, terms_conj)
        mu_conj = compute_mu(ricci_conj)
        graph1 = build_term_overlap_graph(terms1)
        ricci1 = compute_forman_ricci_curvature(graph1, terms1)
        mu1 = compute_mu(ricci1)
        graph2 = build_term_overlap_graph(terms2)
        ricci2 = compute_forman_ricci_curvature(graph2, terms2)
        mu2 = compute_mu(ricci2)
        metric_values.append(mu_conj)
        instances_tested += 1
        if mu_conj - mu1 - mu2 > math.log2(1 + N):
            conjecture_holds = False
            counterexample = f"Regime D: mu_conj={mu_conj} > mu1+mu2+log2(1+N)={mu1+mu2+math.log2(1+N)}"
            break

if not metric_values:
    metric_value = 0.0
else:
    metric_value = sum(metric_values) / len(metric_values)

return {
    "metric_name": "mu",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:

```


#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	claude_max	opus	0	0	7375
5	test_gen	mistral	codestral-latest	0	0	316785
6	test_gen	mistral	codestral-latest	0	0	316900
7	test_gen	mistral	codestral-latest	0	0	314942
8	test_gen	mistral	codestral-latest	0	0	314761
9	critic	claude_max	opus	0	0	4115
10	judge	claude_max	opus	0	0	5931

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1476886 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/b0a4fb5d3039.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b0a4fb5d3039.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b0a4fb5d3039.tar.gz` (if generated)